Big data analytics workflows with Databricks

Lecture 2 — Apache Spark

Andrii Muzychuk

UCU, 2026

POWERED BY APPLIED SCIENCES FACULTY



Lecture 2 — Apache Spark

A unified analytics engine for large-scale data processing

Overview

- Why Spark?
- Review Apache Spark components
- How Apache Spark executes a series of queries
- Spark UI
- Operations in Spark
- Hints to optimise a Spark job

Why Spark?

Apache Spark is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters

Why Spark?

Apache Spark is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters

- Unify the batches and (near) real-time streaming data processing
- Works on structured tables and unstructured data such as JSON or images
- Runs on Hadoop, Kubernetes, standalone, or in the cloud
- In memory caching => faster data processing
- Rich integrations with data science frameworks, BI tools and storage systems
- SQL analytics & Data science at scale
- Machine learning (Java, Scala, Python, and R)











Why Spark?

Apache Spark is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters

- Unify the batches and (near) real-time streaming data processing
- Works on structured tables and unstructured data such as JSON or images
- Runs on Hadoop, Kubernetes, standalone, or in the cloud
- In memory caching => faster data processing
- Rich integrations with data science frameworks, BI tools and storage systems
- SQL analytics & Data science at scale
- Machine learning (Java, Scala, Python, and R)









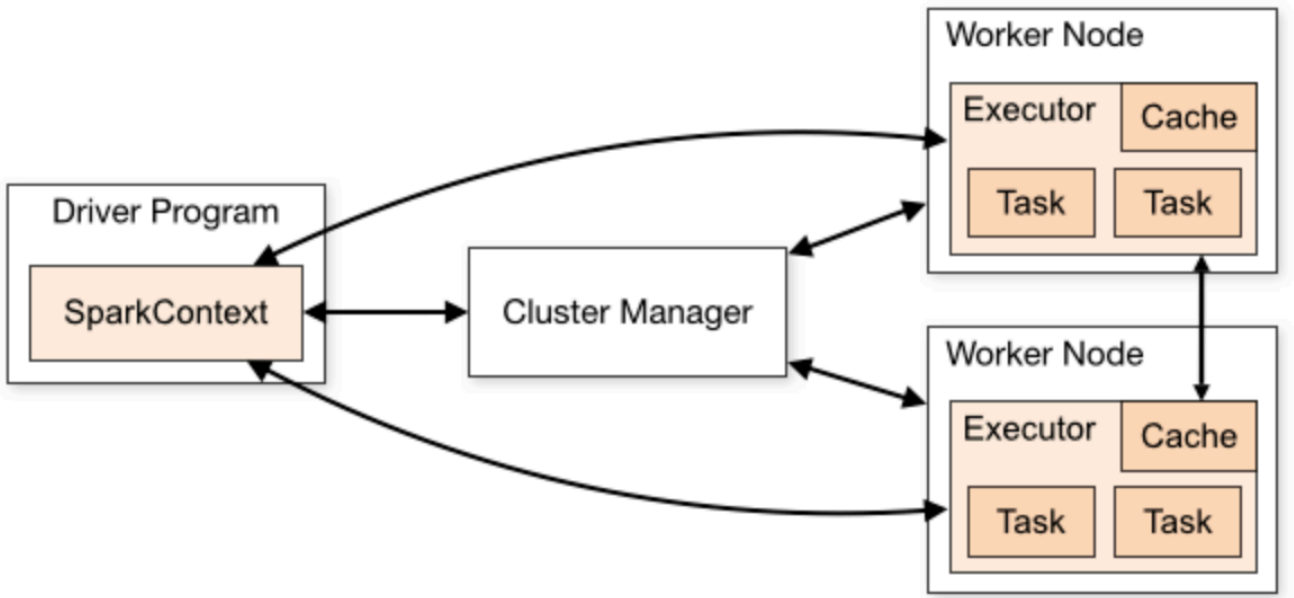


Optimizing Spark jobs for performance remains a significant challenge!

POWERED BY APPLIED SCIENCES FACULTY

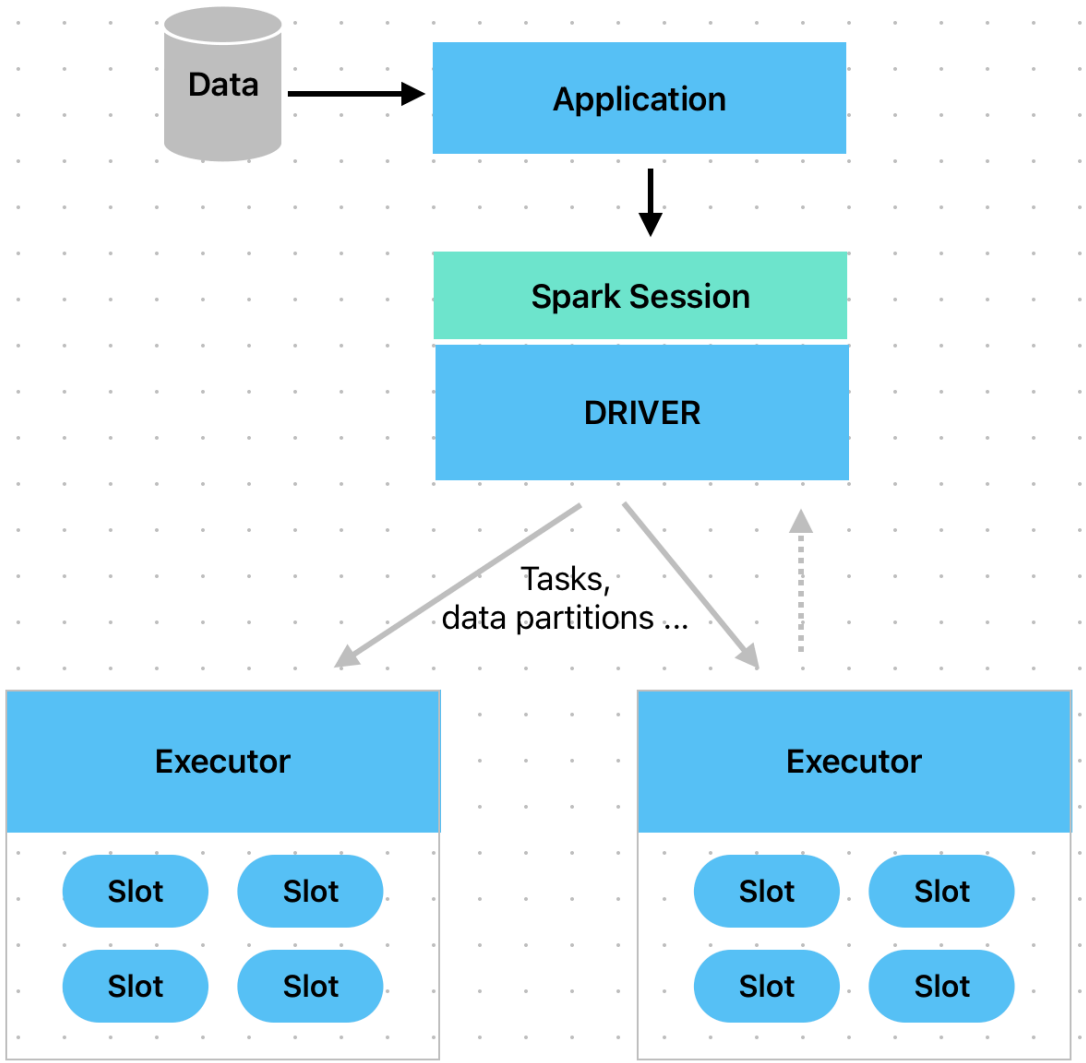
Apache Spark Architecture

Main Components



Key words

How it works (highlevel)



App communicates with driver using spark session

- figures out what is needed
- what is the most efficient way
- splits into tasks

- slot = core
- max tasks in parallel = total # of slots

Key points

- **Cluster** - group of Nodes
- **Nodes** - machines (VMs) within a cluster
- **Driver** - runs the Spark app, coordinates work, assigns tasks to slots. The driver plans jobs and makes sure they are executed. If an executor fails, the driver reassigns its tasks to another one. The driver almost never touches the data itself.
- **Executor** - a JVM process in which tasks run; it executes tasks and reports status. There can be more than one executor per worker node — on Databricks it is always 1:1
- **Slots** / **Core** / **Threads** - lowest unit of parallelization. There can be as many tasks executed by Executor in parallel as many slots
- **Parallelization** - scale horizontally (executors) or vertically (cores)
- **Partition** - chunk of larger dataset
- Application -> job -> stages -> tasks
- Driver and the executors processes run on the Java Virtual Machine

Question

Q: How many tasks can run **in parallel** on a cluster of 2 workers, each with 4 CPU cores?

Question

Q: How many tasks can run **in parallel** on a cluster of 2 workers, each with 4 CPU cores?

A: **8** — one slot per core, 2 x 4 = 8 slots, and a slot runs exactly one task at a time.

Question

Q: How many tasks can run **in parallel** on a cluster of 2 workers, each with 4 CPU cores?

A: **8** — one slot per core, 2 x 4 = 8 slots, and a slot runs exactly one task at a time.

Follow-up: your DataFrame has **200** partitions. How many *waves* of tasks does the stage take?

Question

Q: How many tasks can run **in parallel** on a cluster of 2 workers, each with 4 CPU cores?

A: **8** — one slot per core, 2 x 4 = 8 slots, and a slot runs exactly one task at a time.

Follow-up: your DataFrame has **200** partitions. How many *waves* of tasks does the stage take?

200 / 8 = **25 waves**. Partition count drives the amount of work; slot count drives how much of it happens at once.

Query Execution: Bag of candies example



Can we in 60 seconds:

- filter a specific candy color?
- count how many candies do we have?
- identify distinct candies?

Query Execution: Bag of candies example

What if there are multiple candy bars?



Scenario 1: Filtering

What resource do we need to filter yellow candies?

Scenario 1: Filtering

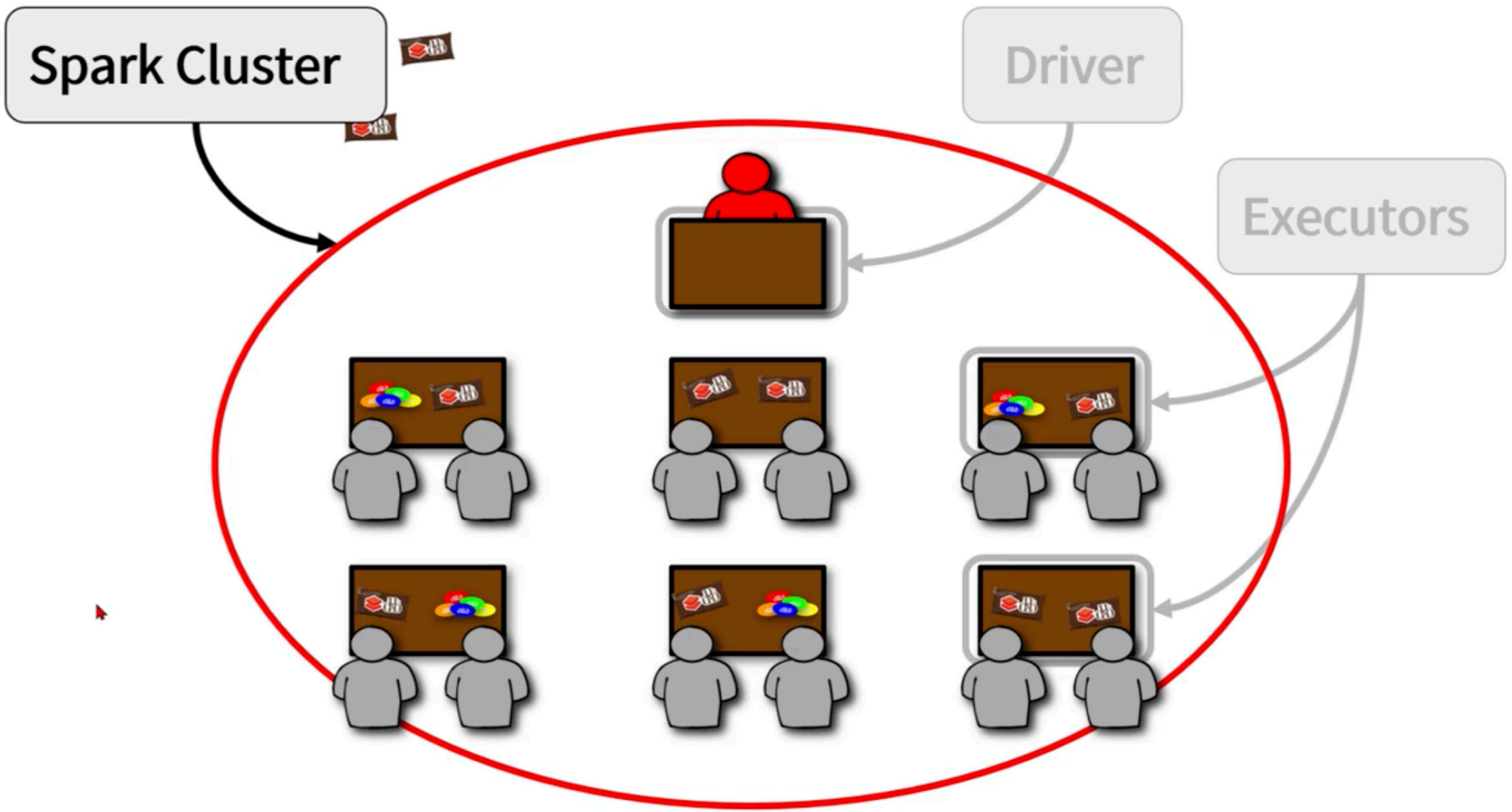
What resource do we need to filter yellow candies?

- [ ] A table or place to work
- [ ] A place from which we can retrieve the candies
- [ ] Students to process the bag
- [ ] An orchestrator
- [ ] Candies properly distributed to bags
- [ ] A place to store non-yellow candies

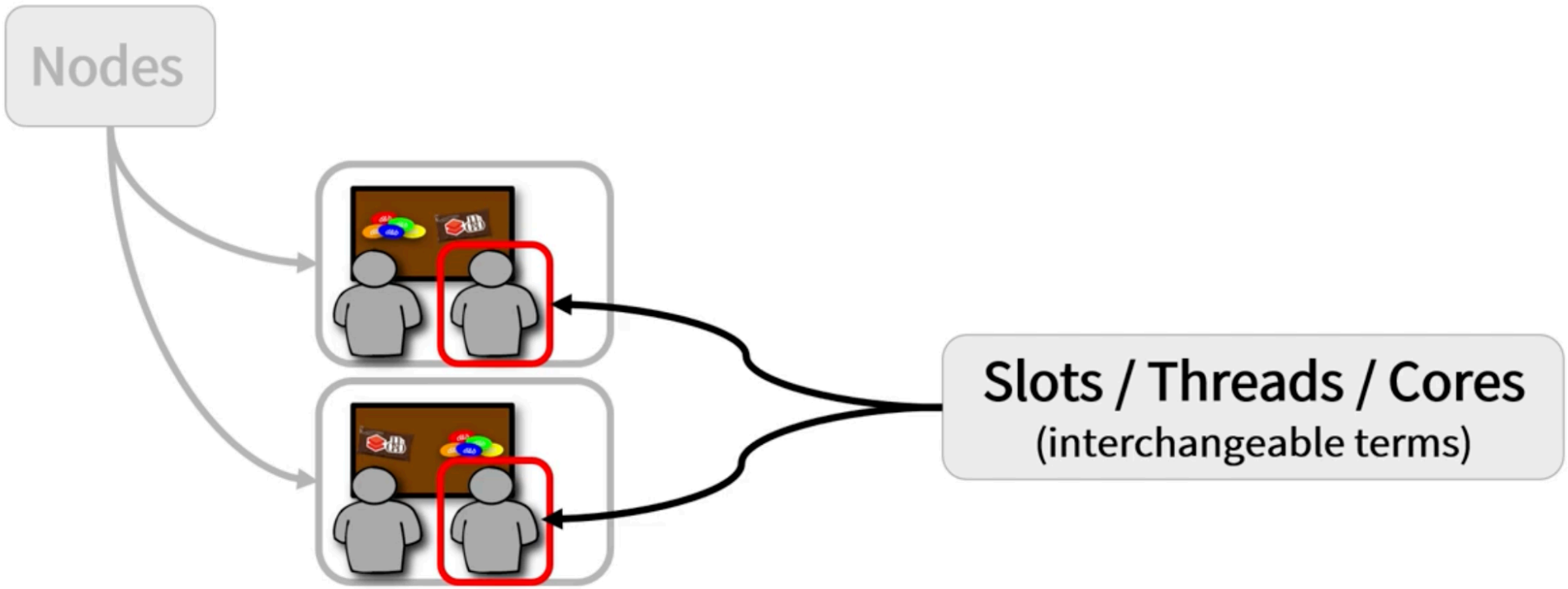
Scenario 1: Filtering

What resource do we need to filter yellow candies?

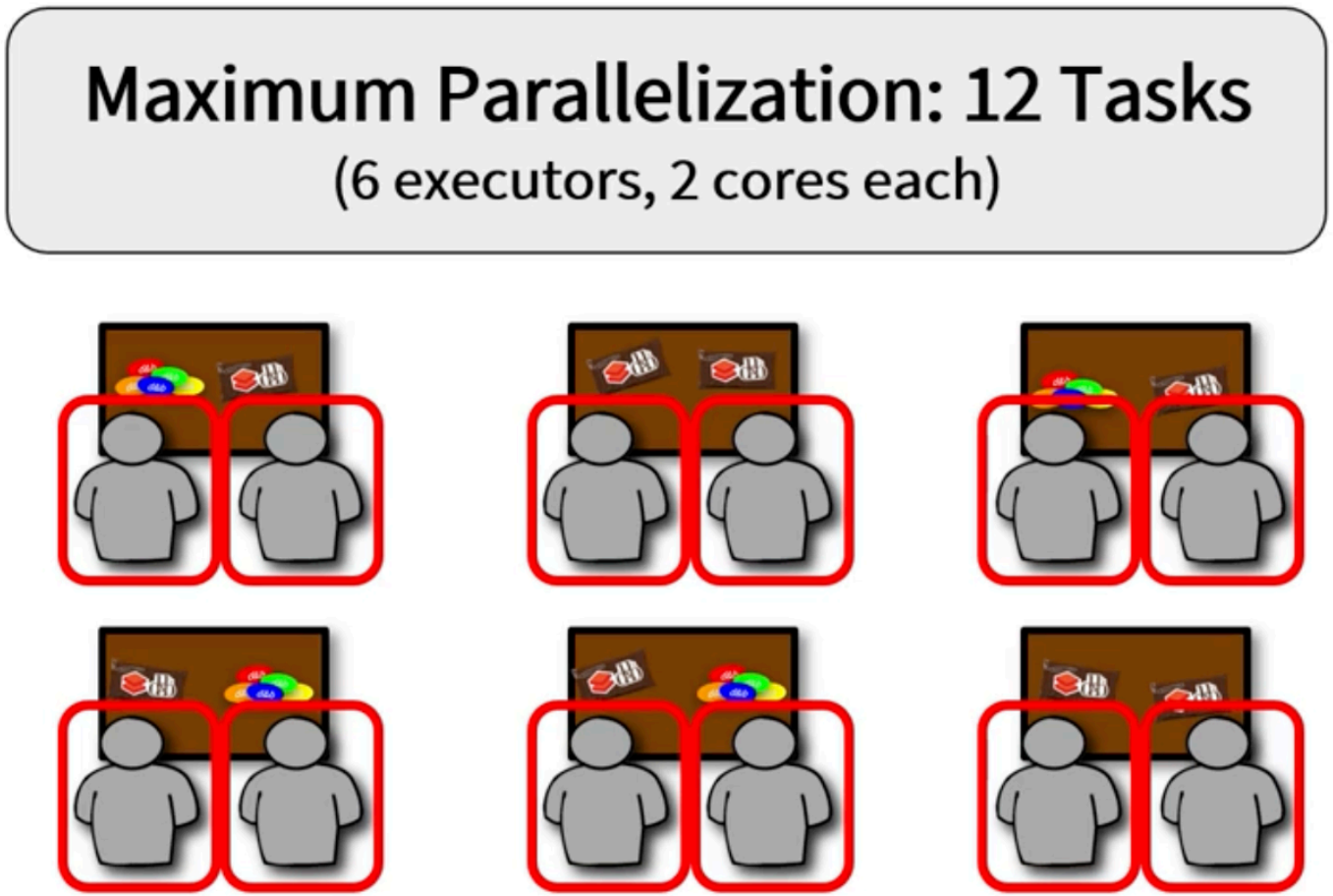
Spark Cluster analogy



Slots analogy



Parallelization analogy



Scenario 2: Counting (analogy)

How many individual pieces of candy do we have in the bag?

Scenario 2: Counting (analogy)

How many individual pieces of candy do we have in the bag?

How many stages do we need ?

Scenario 2: Counting (analogy)

How many individual pieces of candy do we have in the bag?

How many stages do we need ?

- **Stage 1: Local count**
  - Driver can apply speculative execution strategy
  - Instruct each task to persist calculated total and save it for later (data is written to disk)

Scenario 2: Counting (analogy)

How many individual pieces of candy do we have in the bag?

How many stages do we need ?

- **Stage 1: Local count**
  - Driver can apply speculative execution strategy
  - Instruct each task to persist calculated total and save it for later (data is written to disk)
- **Stage 2: Global count**
  - Driver instruct one of executors to grab data from previous stage
  - Final count is produced
  - Result can be send back driver

Scenario 3: Distinct

Which unique kinds of candy do we have in the bag?

Scenario 3: Distinct

Which unique kinds of candy do we have in the bag?

- **Stage 1: Local distinct**
  - identify local distinct candies
  - data is written to disc
- **Stage 2 Global distinct**
  - driver gives task to collect data (e.g. collect all red candies from all of executors) once collected, reduce to unique colors
  - return results to driver

Stage recap

- A stage can not be completed until all tasks are completed
- Spark 3.x can run some stages in parallel
- One long-running task can delay an entire stage from completing
- The shuffle between two stages is one of the most expensive operations in Spark

Fundamental Data structures of Spark

- Resilient Distributed Dataset (RDD)
- DataFrame
- DataSet (Scala/Java only, out of scope)

Resilient Distributed Dataset (RDD)

- it is basic abstraction in Spark (low level API)
- Represents an **immutable** and **partitioned** collection of elements that can be operated on in parallel.
- Provides fault tolerance, data lineage and wide range of transformations and actions for large scaled data processing

Spark DataFrame

- High-level abstraction over RDDs
- Is a distributed collections of data, organized into rows and columns similar to traditional database tables
- Each column has a name and an associated type
- Offer optimization though Spark Catalyst optimizer

Operations in Spark

- Transformation
- Action

Transformation

- Takes one or more RDD / DataFrames as input and produces RDD / DataFrames as output
- Transformation can be either **Narrow** or **Wide**
- Can be viewed as business logic you want to apply to data
- **Transformations are lazy**

Transformation

- Takes one or more RDD / DataFrames as input and produces RDD / DataFrames as output
- Transformation can be either **Narrow** or **Wide**
- Can be viewed as business logic you want to apply to data
- **Transformations are lazy**

Narrow transformations:

- E.g.: Filter, Select, Cast, Create new column
- Where there is no need to exchange the data between executors

Transformation

- Takes one or more RDD / DataFrames as input and produces RDD / DataFrames as output
- Transformation can be either **Narrow** or **Wide**
- Can be viewed as business logic you want to apply to data
- **Transformations are lazy**

Narrow transformations:

- E.g.: Filter, Select, Cast, Create new column
- Where there is no need to exchange the data between executors

Wide transformations:

Actions

- E.g.: collect, count, show, take
- When we demand for the result
- Operation that kicks off the computation

Transformation tips

- If possible, filter data first.
- Try to group wide transformations together for best automatic optimization
- Aim for:

  Narrow -> Narrow -> Wide -> Wide -> Wide -> Narrow
- Avoid :

  Narrow -> Wide -> Narrow -> Wide -> Narrow -> Wide

Shuffle recap

- At a high level, shuffle is a process of moving data through the network so we can proceed with the next stage
- Occurs during wide transformations
- One of the most significant, yet often unavoidable, cause of performance degradation

Shuffle recap

- At a high level, shuffle is a process of moving data through the network so we can proceed with the next stage
- Occurs during wide transformations
- One of the most significant, yet often unavoidable, cause of performance degradation

Q: Is shuffle the only thing that can slow down a Spark job?

Meet the Spill

Shuffle is the intentional redistribution and movement of data across different partitions and nodes, while spill is an emergency overflow process that happens when data exceeds available RAM and writes to local disk

- **Spill Definition**: The fallback action of writing temporary in-memory data to local disk when execution memory fills up.
- **Purpose**: Prevents out-of-memory errors when working with large or skewed data partitions.
- **Nature**: Unplanned performance bottleneck caused by resource constraints

Performace impact

High Shuffle: Normal for complex analytical queries, but can be optimized via better partitioning or filtering.

Performace impact

High Shuffle: Normal for complex analytical queries, but can be optimized via better partitioning or filtering.

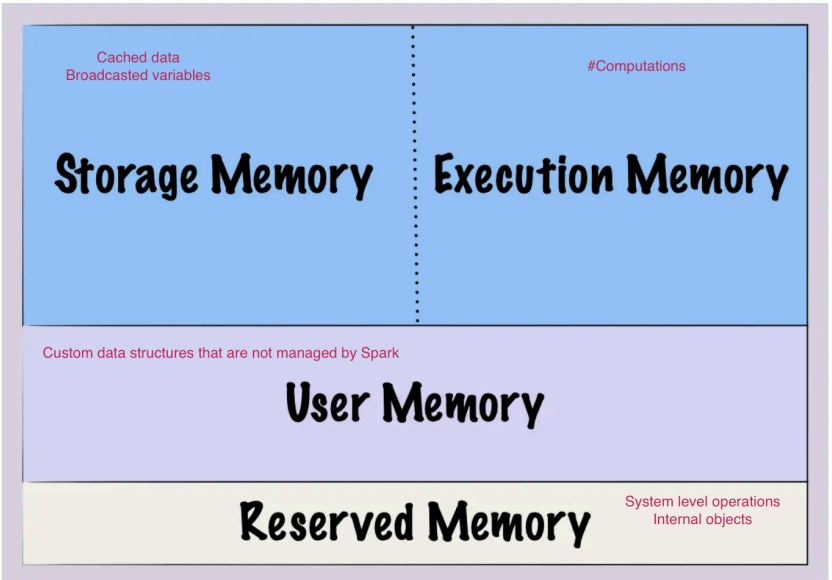
High Spill: A performance killer that slows down jobs due to heavy disk I/O; fix it by increasing executor memory, optimizing data skew, or lowering partition sizes.

Spark Optimization

Built-in Optimization

- Catalyst optimizer - generate optimized execution plan
- Tungsten - generate optimized code, using Whole-stage Codegen and improve how the data are handled in the cluster
- Adaptive Query Execution (AQE) - dynamic re-optimization of the execution plan at runtime

Spark memory management



- **Reserved** - fixed slice kept for Spark's own internals
- **Execution** - shuffles, joins, sorts, aggregations. Spilled to disk when it runs out
- **Storage** - cached DataFrames and broadcast variables
- **User** - your data structures, UDF state, anything you allocate yourself

Execution and storage share one region and borrow from each other — a large cache can starve a shuffle, which is why `OutOfMemory` errors often show up right after a `.ca`

Job optimization Hints

- Choose the **right file format** to store data
- Data partitioning can help
- Cache DataFrames and intermediate results
- Minimize volume of data by applying filters
- Avoid imbalance in size between the different partitions
- Avoid expensive operations like sort
- Avoid UDFs
- Maximise parallelism in Spark
- Beware of **shuffle operations**

Thank you :)